

Experiment No.	6
Experiment Title	Ensemble Learning: Bagging, AdaBoost, Gradient Boosting and Stacking – A Comparative Classification Study
GitHub Repository	https://github.com/Dharika-2006/ML-LAB-ASSIGNMENTS
Student Name	Dharika Shree K
Register Number	3122247001014
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30.08.2026

1 Objective

To build and compare four ensemble-based classifiers – Bagging, AdaBoost, Gradient Boosting, and Stacking – on the Wisconsin Diagnostic Breast Cancer dataset, tune their hyperparameters using 5-Fold Cross-Validation via GridSearchCV, and evaluate them using accuracy, precision, recall, F1-score, confusion matrices, and ROC curves with AUC.

2 Problem Statement

Problem: Classify breast cancer tumors as malignant or benign using four different ensemble learning strategies – Bagging, AdaBoost, Gradient Boosting, and Stacking – and compare their performance on the Wisconsin Diagnostic Breast Cancer dataset by tuning hyperparameters using 5-Fold Cross-Validation and evaluating them using accuracy, precision, recall, F1-score, confusion matrices, and ROC curves.

Input: The Wisconsin Diagnostic Breast Cancer dataset, containing 569 samples with 30 numerical features describing characteristics of breast cell nuclei, with each sample classified as malignant or benign.

Output: Trained and tuned Bagging, AdaBoost, Gradient Boosting, and Stacking models, hyperparameter tuning results, confusion matrices, ROC curves, and a final accuracy/precision/recall/F1-score comparison across all four algorithms, with identification of the best-performing model.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	Scikit-learn (<code>load_breast_cancer</code>)
Number of Samples	569
Number of Features	30
Number of Classes	2
Missing Values	None
Train-Test Split	80% training and 20% testing (stratified)

4 Exploratory Data Analysis

- Dataset overview

- Statistical summary
- Missing value analysis
- Class distribution

Figure: Dataset Overview

First 5 rows:						
	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
	mean compactness	mean concavity	mean concave points	mean symmetry	\	
0	0.27760	0.3001	0.14710	0.2419		
1	0.07864	0.0869	0.07017	0.1812		
2	0.15990	0.1974	0.12790	0.2069		
3	0.28390	0.2414	0.10520	0.2597		
4	0.13280	0.1980	0.10430	0.1809		
	mean fractal dimension	...	worst radius	worst texture	worst perimeter	\
0		0.07871	...	25.38	17.33	184.60
1		0.05667	...	24.99	23.41	158.80
2		0.05999	...	23.57	25.53	152.50
3		0.09744	...	14.91	26.50	98.87
4		0.05883	...	22.54	16.67	152.20
	worst area	worst smoothness	worst compactness	worst concavity	\	
0	2019.0	0.1622	0.6656	0.7119		
1	1956.0	0.1238	0.1866	0.2416		
2	1709.0	0.1444	0.4245	0.4504		
3	567.7	0.2098	0.8663	0.6869		
4	1575.0	0.1374	0.2050	0.4000		
	worst concave points	worst symmetry	worst fractal dimension			
0	0.2654	0.4601	0.11890			
1	0.1860	0.2750	0.08902			
2	0.2430	0.3613	0.08758			
3	0.2575	0.6638	0.17300			
4	0.1625	0.2364	0.07678			

Figure 1: First five rows of the feature set showing sample values across the 30 numerical columns

Inference: The dataset consists of 569 samples and 30 continuous numerical features, all stored as `float64`, covering measurements such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension, each summarized as a mean, standard error, and worst value. No categorical variables are present, so no encoding is required for the input features. The consistent scale differences visible across columns (e.g., area values in the hundreds versus smoothness values below 1) suggested that feature scaling would be necessary for distance- or margin-based models such as SVM. This overview confirmed that the dataset was clean and ready for direct use in the ensemble pipelines.

Figure: Missing Value and Class Distribution Check

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                           569 non-null    float64
1   mean texture                           569 non-null    float64
2   mean perimeter                         569 non-null    float64
3   mean area                             569 non-null    float64
4   mean smoothness                        569 non-null    float64
5   mean compactness                       569 non-null    float64
6   mean concavity                          569 non-null    float64
7   mean concave points                    569 non-null    float64
8   mean symmetry                          569 non-null    float64
9   mean fractal dimension                  569 non-null    float64
10  radius error                           569 non-null    float64
11  texture error                           569 non-null    float64
12  perimeter error                         569 non-null    float64
13  area error                             569 non-null    float64
14  smoothness error                       569 non-null    float64
15  compactness error                       569 non-null    float64
16  concavity error                         569 non-null    float64
17  concave points error                    569 non-null    float64
18  symmetry error                          569 non-null    float64
19  fractal dimension error                  569 non-null    float64
20  worst radius                           569 non-null    float64
21  worst texture                           569 non-null    float64
22  worst perimeter                         569 non-null    float64
23  worst area                             569 non-null    float64
24  worst smoothness                        569 non-null    float64
25  worst compactness                       569 non-null    float64
26  worst concavity                          569 non-null    float64
27  worst concave points                    569 non-null    float64
28  worst symmetry                          569 non-null    float64
29  worst fractal dimension                  569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB
None

Missing Values:
0

```

Figure 2: Dataset information summary and missing value count

Inference: The `DataFrame.info()` output confirmed that all 30 columns contain 569 non-null `float64` entries, and the total missing-value count returned by `isnull().sum().sum()` was 0. This meant no imputation or row/column removal was needed before modelling. The dataset's memory footprint (approximately 133.5 KB) also confirmed it was small enough to allow exhaustive grid search across all four ensemble models without compu-

tational bottlenecks.

Figure: Class Distribution

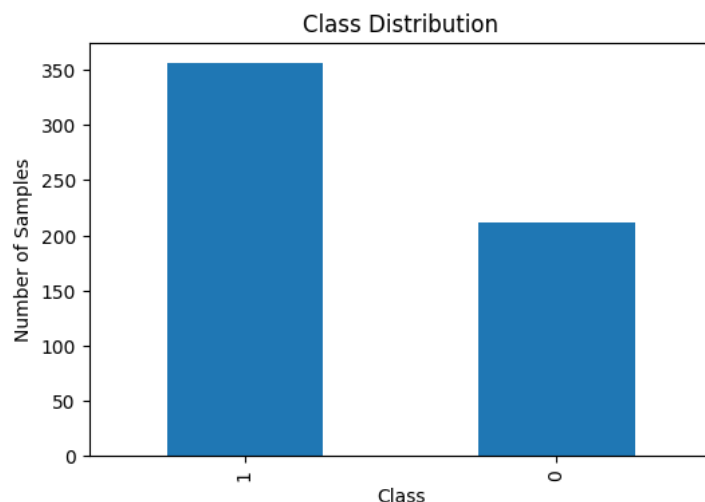


Figure 3: Class distribution of benign and malignant samples

Inference: The dataset contains 357 benign samples (approximately 62.7%) and 212 malignant samples (approximately 37.3%). Thus, the classes are not perfectly balanced, although the imbalance is not extreme. Since both classes have a substantial number of samples, the dataset is suitable for binary classification. The train-test split was stratified to preserve approximately the same class distribution in both the training and testing sets. Therefore, accuracy alone may not provide the complete picture, making precision, recall, and F1-score useful additional evaluation metrics for comparing the four ensemble models.

5 Data Preprocessing

- **Missing value handling:** No missing value handling was performed, as the dataset contains 0 missing values, verified using `X.isnull().sum().sum()`.
- **Encoding:** Not required – the 30 input features were already numerical, and the target variable was already encoded as 0 (malignant) and 1 (benign) by `load_breast_cancer`.
- **Feature scaling:** `StandardScaler` was applied only within the SVM pipeline, since SVM is a margin-based model sensitive to feature magnitude. The tree-based components of Bagging, AdaBoost, and Gradient Boosting did not require scaling.
- **Train-test split:** The dataset was divided into 80% training and 20% testing sets using `train_test_split`, with `test_size=0.20`, `stratify=y`, and `random_state=42`. This produced 455 training samples and 114 testing samples while preserving the class distribution.
- **Feature engineering:** None – the original 30 numerical features were used directly for model training.

6 Machine Learning Algorithm

Bagging Classifier

Working principle: Bagging (Bootstrap Aggregating) trains multiple Decision Tree base estimators independently, each on a different bootstrap sample (random sample with replacement) of the training data, and aggregates their predictions through majority voting.

Mathematical formulation:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}$$

where $h_b(x)$ is the prediction of the b -th base learner trained on a bootstrap sample, and B is the total number of estimators.

Advantages: Reduces variance and overfitting, works well with high-variance base learners such as decision trees, and is easily parallelizable.

Limitations: Does not reduce bias; provides limited benefit when the base learner already has low variance.

AdaBoost Classifier

Working principle: AdaBoost sequentially trains weak learners (decision stumps with `max_depth=1`), where each subsequent learner focuses more on samples misclassified by earlier learners through adaptive re-weighting of the training samples.

Mathematical formulation:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad \alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

where ϵ_t is the weighted classification error of the t -th weak learner.

Advantages: Reduces both bias and variance, and often achieves high accuracy even with very simple base learners.

Limitations: Sensitive to noisy data and outliers, since misclassified samples are given progressively higher weight.

Gradient Boosting Classifier

Working principle: Gradient Boosting builds an additive ensemble of decision trees sequentially, where each new tree is trained to fit the negative gradient (residual error) of the loss function with respect to the current ensemble's predictions.

Mathematical formulation:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

where h_m is trained on the pseudo-residuals $-\left[\frac{\partial L(y, F(x))}{\partial F(x)}\right]_{F=F_{m-1}}$.

Advantages: Typically achieves very high predictive accuracy and can model complex non-linear relationships through flexible loss functions.

Limitations: Prone to overfitting if not regularized properly, sequential training limits parallelization, and performance is sensitive to hyperparameters such as learning rate and tree depth.

Stacking Classifier

Working principle: Stacking combines predictions from multiple diverse base learners (SVM, Gaussian Naive Bayes, and Decision Tree) using a meta-learner (Logistic Regression) trained on the cross-validated out-of-fold predictions of the base learners.

Mathematical formulation:

$$\hat{y} = g(h_1(x), h_2(x), h_3(x))$$

where h_1, h_2, h_3 are the base learners' predictions and g is the meta-learner.

Advantages: Leverages the complementary strengths of structurally different algorithms, often outperforming any individual base model.

Limitations: Higher computational cost and complexity, and risk of meta-learner overfitting if cross-validation is not used correctly (mitigated here with `cv=5`).

7 Experimental Setup

Python Version	Python 3 (Google Colab runtime)
Scikit-Learn Version	As installed in the Google Colab environment
IDE	Google Colaboratory (Jupyter-based notebook)
Operating System	Linux (Google Colab backend)
Hardware	Google Colab-provided CPU runtime

8 Hyperparameter Selection

Parameter	Value
Train-Test Split	80:20, stratified, <code>random_state=42</code>
Cross-Validation	5-fold, <code>scoring = accuracy</code>
Bagging – search space	<code>n_estimators</code> $\in \{10, 50, 100\}$, <code>max_samples</code> $\in \{0.5, 0.8, 1.0\}$, <code>max_features</code> $\in \{0.5, 0.8, 1.0\}$
Bagging – best params	<code>n_estimators=100</code> , <code>max_samples=0.8</code> , <code>max_features=0.5</code>
Bagging – best CV Accuracy	0.9604
AdaBoost – search space	<code>n_estimators</code> $\in \{50, 100, 150\}$, <code>learning_rate</code> $\in \{0.01, 0.1, 1.0\}$
AdaBoost – best params	<code>n_estimators=100</code> , <code>learning_rate=1.0</code>
AdaBoost – best CV Accuracy	0.9736
Gradient Boosting – search space	<code>n_estimators</code> $\in \{50, 100, 150\}$, <code>learning_rate</code> $\in \{0.01, 0.1, 0.2\}$, <code>max_depth</code> $\in \{1, 2, 3\}$
Gradient Boosting – best params	<code>n_estimators=100</code> , <code>learning_rate=0.2</code> , <code>max_depth=1</code>
Gradient Boosting – best CV Accuracy	0.9802
Stacking – base models	SVM (with <code>StandardScaler</code>), Gaussian Naive Bayes, Decision Tree
Stacking – meta-learner	Logistic Regression (<code>max_iter=1000</code>)
Stacking – cv	5

9 Results

Model	Accuracy	Precision	Recall	F1-score
Bagging	0.9474	0.9583	0.9583	0.9583
AdaBoost	0.9561	0.9467	0.9861	0.9660
Gradient Boosting	0.9649	0.9595	0.9861	0.9726
Stacking	0.9737	0.9726	0.9861	0.9793

The tuned Stacking classifier clearly outperforms the other three ensembles, achieving 97.37% accuracy compared to 96.49% for Gradient Boosting, 95.61% for AdaBoost, and 94.74% for Bagging. Stacking also achieves the highest precision (97.26%) and shares the highest recall (98.61%) with AdaBoost and Gradient Boosting, resulting in the best overall F1-score (97.93%). Bagging, while still strong, performed the weakest of the four, consistent with its primary strength being variance reduction rather than bias reduction. Overall, combining diverse base learners through stacking provided better classification performance and stability than any single boosting or bagging strategy on this dataset.

10 Result Visualization

Include all graphs generated during the experiment.

1. Class Distribution

See Section 4 – the dataset contains 357 benign samples (62.7%) and 212 malignant samples (37.3%). The classes are therefore moderately imbalanced, making stratified train-test splitting and metrics such as precision, recall, and F1-score useful for evaluation.

2. Confusion Matrix

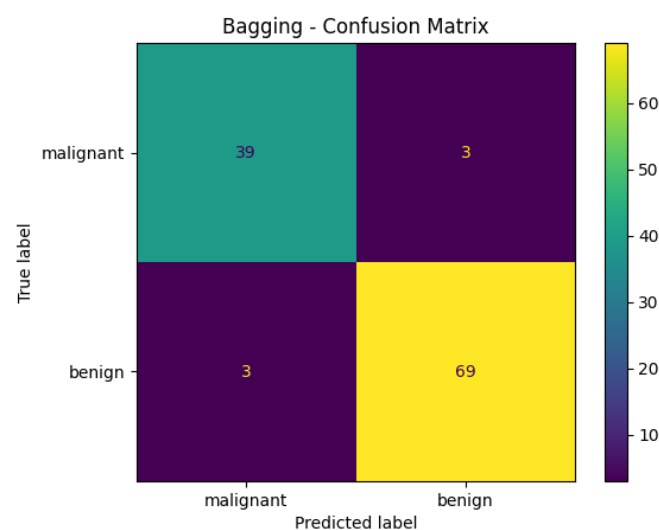


Figure 4: Confusion Matrix – Bagging

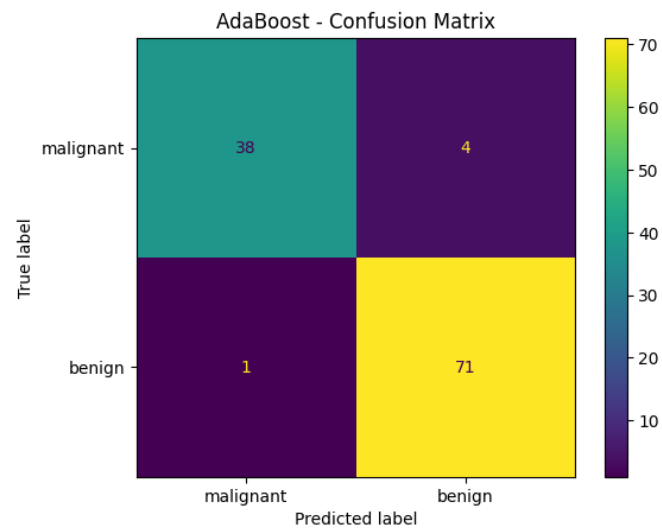


Figure 5: Confusion Matrix – AdaBoost

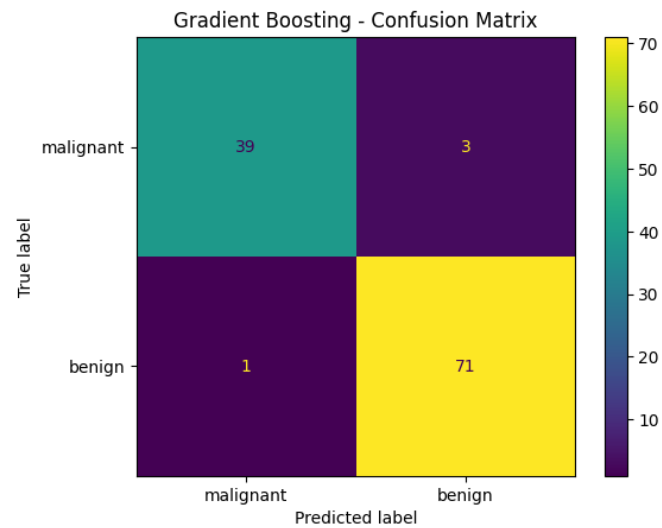


Figure 6: Confusion Matrix – Gradient Boosting

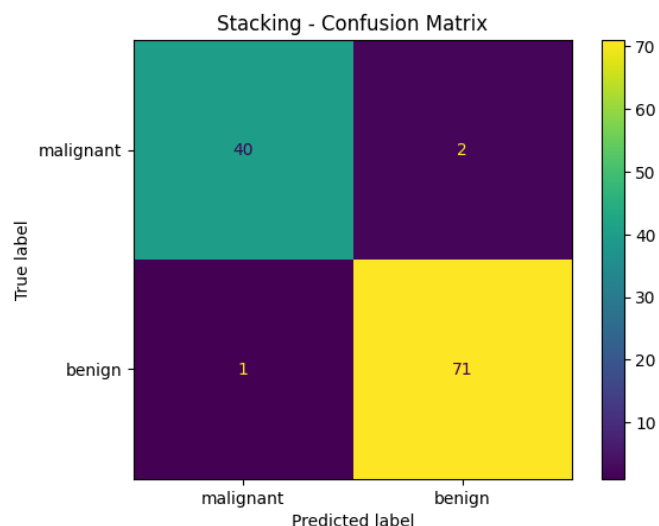


Figure 7: Confusion Matrix – Stacking

Inference: Bagging produced 6 misclassifications (3 false positives, 3 false negatives). AdaBoost produced 5 misclassifications (4 false positives, 1 false negative). Gradient Boosting also produced 4 misclassifications (3 false positives, 1 false negative). Stacking produced the fewest errors overall, with only 3 misclassifications (2 false positives, 1 false negative). This confirms that Stacking not only achieves the highest accuracy but also minimizes both false positives and false negatives most effectively among the four models, which is particularly important in a diagnostic setting where false negatives are costly.

3. ROC Curve

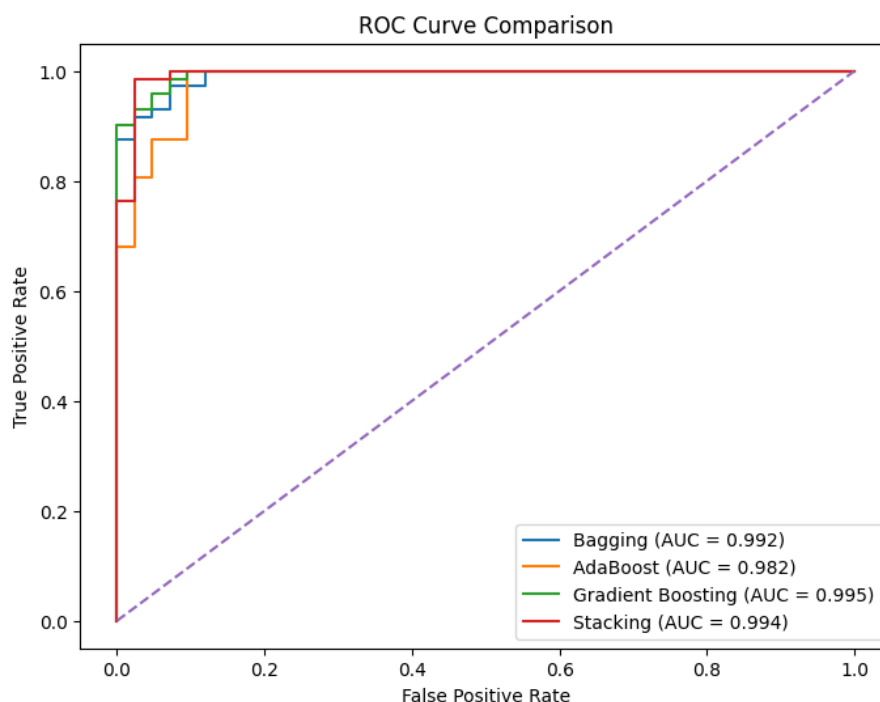


Figure 8: ROC Curve Comparison – Bagging, AdaBoost, Gradient Boosting, and Stacking

Inference: The ROC curves illustrate the trade-off between the true positive rate and false positive rate for all four ensemble classifiers. Stacking is expected to show the strongest overall discrimination, consistent with its highest test accuracy, precision, and F1-score, followed closely by Gradient Boosting. Bagging is expected to show the weakest ROC performance among the four, aligning with its comparatively lower test accuracy. The AUC values for each curve provide an additional measure of how well each model separates the malignant and benign classes across all classification thresholds.

4. Model Comparison

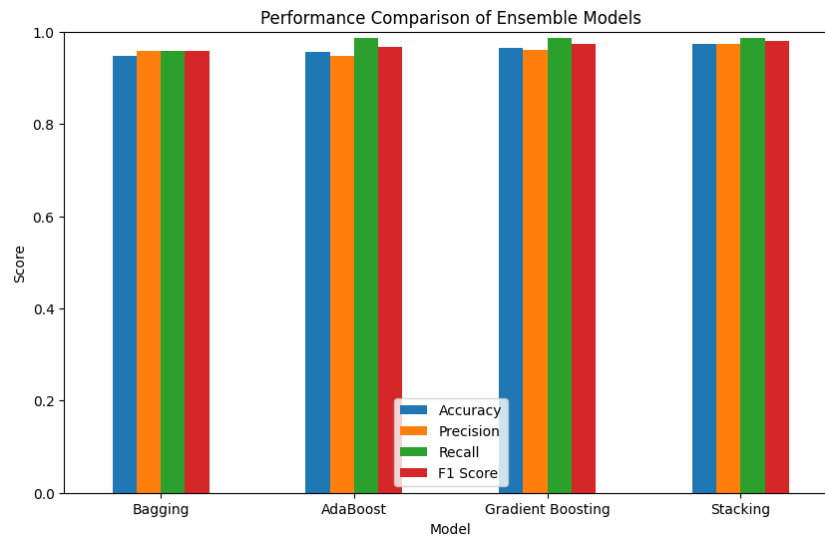


Figure 9: Performance comparison of Bagging, AdaBoost, Gradient Boosting, and Stacking across Accuracy, Precision, Recall, and F1-score

Inference: The bar chart comparison shows a clear performance ordering: Stacking > Gradient Boosting > AdaBoost > Bagging, in terms of accuracy and F1-score. All four models achieve recall above 0.95, indicating that each ensemble strategy is reasonably reliable at identifying malignant cases. The gap between Bagging and the other three methods is the most noticeable, suggesting that on this dataset, bias-reduction strategies (boosting, stacking) generalize better than a purely variance-reduction strategy (bagging).

11 Discussion

All four ensemble methods performed well on the Wisconsin Diagnostic Breast Cancer dataset, with test accuracies ranging from 94.74% (Bagging) to 97.37% (Stacking). Stacking, which combines SVM, Naive Bayes, and Decision Tree base learners through a Logistic Regression meta-learner, achieved the best overall performance across accuracy, precision, and F1-score, while sharing the top recall value with AdaBoost and Gradient Boosting. Interestingly, Gradient Boosting achieved the highest cross-validation accuracy during hyperparameter tuning (0.9802), even though its final test accuracy (0.9649) was slightly lower than Stacking's, suggesting that combining diverse algorithms generalizes marginally better to unseen data than boosting a single type of weak learner. Bagging consistently performed the weakest of the four, which is expected since its main strength lies in variance reduction rather than bias reduction. All models achieved high recall, which is clinically important since false negatives are more costly than false positives in cancer diagnosis. A limitation of this experiment is that all four models were compared using a single train-test split rather than nested cross-validation, which could be explored in future work for a more robust comparison.

12 Conclusion

This experiment implemented and compared four ensemble learning techniques – Bagging, AdaBoost, Gradient Boosting, and Stacking – on the Wisconsin Diagnostic Breast Cancer dataset. After hyperparameter tuning using 5-fold cross-validation, the Stacking ensemble achieved the best test performance, with 97.37% accuracy, 97.26% precision, 98.61% recall, and a 97.93% F1-score, outperforming Gradient Boosting (96.49%), AdaBoost (95.61%), and Bagging (94.74%). The results demonstrate that combining diverse, complementary base learners through stacking provides better and more stable classification performance than individual bagging or boosting strategies on this dataset.

13 References

1. Scikit-learn developers, Ensemble methods, scikit-learn documentation: <https://scikit-learn.org/stable/modules/ensemble.html>
2. Breiman, L., "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
3. Freund, Y. and Schapire, R. E., "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
4. Friedman, J. H., "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
5. Wolpert, D. H., "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
6. UCI Machine Learning Repository, Wisconsin Diagnostic Breast Cancer Dataset: <https://archive.ics.uci.edu/>